

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
PROGRAMA DE PÓS-GRADUAÇÃO EM COMPUTAÇÃO

IGOR FÁBIO STEINMACHER

**Técnicas de *Web Caching* e *Prefetching*
com Prioridades**

Trabalho Individual I
TI-XXX

Prof. Dr. José Valdeni de Lima
Orientador

Porto Alegre, Fevereiro de 2004

SUMÁRIO

LISTA DE ABREVIATURAS.....	4
LISTA DE FIGURAS.....	5
LISTA DE FIGURAS.....	6
RESUMO.....	7
ABSTRACT.....	8
1. INTRODUÇÃO.....	9
2. WEB CACHING	11
2.1. Histórico.....	11
2.2. <i>Web Caching</i> : Usar ou Não Usar?.....	12
2.3. Propriedades Desejáveis a um Sistema de <i>Web Caching</i>	15
2.3.1. Rapidez no Acesso.....	15
2.3.2. Robustez.....	15
2.3.3. Transparência.....	15
2.3.4. Escalabilidade.....	16
2.3.5. Eficiência.....	16
2.3.6. Adaptatividade.....	17
2.3.7. Estabilidade.....	17
2.3.8. Balanceamento de Carga.....	17
2.3.9. Simplicidade.....	17
2.4. Medindo a Performance de sistemas de <i>Web Caching</i>	17
2.5. Considerações Finais.....	18
3. TIPOS DE WEB CACHES	19
3.1. <i>Browser Cache</i>	19
3.2. <i>Proxy Cache</i>	20
3.3. <i>Proxies</i> de Intercepção (<i>Transparent Proxy Cache</i>).....	21
3.4. <i>Proxies</i> Reversos (<i>Surrogates</i>).....	21
3.5. Considerações Finais.....	22
4. ARQUITETURAS PARA WEB CACHING	23
4.1. Arquitetura Hierárquica.....	23
4.2. Arquitetura Distribuída.....	24
4.3. Arquitetura Híbrida.....	25
4.4. Considerações Finais.....	26

5.	POLÍTICAS DE SUBSTITUIÇÃO	27
5.1.	Estratégias Baseadas nos Mais Recentes	27
5.1.1.	LRU (<i>Least Recently Used</i>).....	27
5.1.2.	LRU-threshold.....	28
5.1.3.	LRU-Min.....	28
5.1.4.	SIZE	28
5.1.5.	Pitkow/Recker	28
5.1.6.	HLRU (<i>History-LRU</i>)	28
5.2.	Estratégias Baseadas em Frequência.....	29
5.2.1.	LFU (<i>Least Frequently Used</i>)	29
5.2.2.	LFU-Aging	29
5.2.3.	LFU-DA (<i>LFU-Dynamic Aging</i>).....	30
5.3.	Estratégias Baseadas nos Mais Recentes/Freqüentes.....	30
5.3.1.	SLRU (<i>Segmented LRU</i>).....	30
5.3.2.	LRU*.....	30
5.3.3.	LRU-Hot	31
5.3.4.	HYPER-G.....	31
5.4.	Estratégias Baseadas em Função	31
5.4.1.	GD-Size (<i>GreedyDual - Size</i>).....	32
5.4.2.	Bolot/Hoschka	32
5.4.3.	HYBRID	32
5.5.	Estratégias Randômicas	33
5.5.1.	Rand	33
5.5.2.	Harmonic.....	33
5.6.	Considerações Finais	33
6.	CONSISTÊNCIA DOS OBJETOS	35
6.1.	Comandos HTTP que auxiliam na consistência dos objetos.....	35
6.2.	Mecanismos de Consistência do <i>Cache</i>	36
6.2.1.	Client-validation	36
6.2.2.	<i>TTL (Time-to-Live)</i>	36
6.2.3.	<i>Alex Protocol (TTL Adaptativa)</i>	36
6.2.4.	Piggyback Client Validation (PCV)	37
6.2.5.	Server-Invalidation	37
6.2.6.	Piggyback Server Invalidation (PSI)	38
6.3.	Considerações Finais	38
7.	BUSCA ANTECIPADA DE INFORMAÇÃO	39
7.1.	Classificação de acordo com o contexto <i>Web</i>.....	40
7.1.1.	Busca Antecipada Baseada no Cliente.....	40
7.1.2.	Busca Antecipada Baseada no <i>Proxy</i>	40
7.1.3.	Busca Antecipada Baseada no Servidor	41
7.1.4.	Busca Antecipada Baseada no <i>Proxy</i> e no Servidor.....	42
7.2.	Classificação de acordo com a Estratégia de Predição.....	42
7.2.1.	Busca Antecipada Baseada em Preferências Pessoais do Usuário.....	43
7.2.2.	Busca Antecipada com Predição Não-Estatística.....	44
7.2.3.	Busca Antecipada com Predição Estatística.....	44
7.3.	Medindo a Performance de Sistemas de Busca Antecipada	45
7.4.	Considerações Finais	46
8.	CONCLUSÃO	48
9.	BIBLIOGRAFIA	50

LISTA DE ABREVIATURAS

LRU	<i>Least Recently Used</i>
LFU	<i>Least Frequently Used</i>
HLRU	<i>History - Least Recently Used</i>
SLRU	<i>Segmented - Least Recently Used</i>
LFU-DA	<i>Least Frequently Used – Dynamic Aging</i>
GD-Size	<i>Greedy-Dual Size</i>
TTL	<i>Time To Live</i>
PCV	<i>Piggyback Client Validation</i>
PSI	<i>Piggyback Server Invalidation</i>
HTTP	<i>Hypertext Transfer Protocol</i>
ISP	<i>Internet Service Provider</i>
PC	<i>Personal Computer</i>
PDA	<i>Personal Digital Assistant</i>
HR	<i>Hit Ratio</i>
BHR	<i>Byte Hit Ratio</i>
CARP	<i>Cache Array Resolution Protocol</i>
ICP	<i>Internet Cache Protocol</i>
RTT	<i>Round Trip Time</i>
PPM	<i>Prediction by Partial Matching</i>
HTML	<i>Hypertext Markup Language</i>
IA	<i>Inteligência Artificial</i>
MFHS	<i>Miss Free Hoard Size</i>

LISTA DE FIGURAS

Figura 2.1 - Latência de uma requisição ao servidor de origem	13
Figura 3.1 – Funcionamento básico de um <i>Browser Cache</i>	20
Figura 3.2 – Funcionamento básico de um <i>Proxy Cache</i>	20
Figura 3.3 – Funcionamento de um <i>Proxy</i> de Intercepção	21
Figura 3.4 – Funcionamento de um <i>Proxy</i> Reverso	22
Figura 4.1 – Estrutura hierárquica, mostrando o posicionamento dos <i>caches</i>	24

LISTA DE FIGURAS

Tabela 5.1 – Vantagens e Desvantagens das estratégias de substituição da *cache* . 34

Tabela 6.1 – Comandos HTTP que auxiliam nos mecanismos de consistência 35

RESUMO

Este trabalho apresenta uma revisão bibliográfica visando a fundamentação teórica de *Web caching* e busca antecipada de informação – ou *prefetching*. O objetivo desta revisão é a definição de uma estratégia de busca antecipada de informação aliada a um sistema de *Web caching* para aumentar a disponibilidade de sistemas que operam em modo desconectado ou semi-conectado. Existe um grande número de pesquisas sendo realizadas visando maior escalabilidade dos serviços *Web*, diminuição do tráfego da rede e a da carga dos servidores, proporcionando aos usuários tempos de resposta cada vez menores. Porém, quando se fala de operação em modo desconectado, apenas um *Web cache* não é interessante, pois a desconexão pode se dar de forma total, isto é, o cliente atuando em *standalone*. Neste caso se faz necessária a utilização de técnicas de busca antecipada de informações. Esta busca se dá no sentido do *cache* buscar antecipadamente os objetos que serão requisitados no futuro, deixando-os disponíveis antes mesmo de haver uma solicitação. Estas técnicas diminuem ainda mais a latência percebida pelo usuário, além de deixar transparente o estado de sua conexão. O uso de busca antecipada na *Web* varia de acordo com o contexto da aplicação que o *cache* está atendendo. Neste sentido serão estudadas algumas das técnicas encontradas na literatura, analisando as suas vantagens e desvantagens.

Palavras-chave: *Web caching*, *prefetching*, modo desconectado.

Title: “Web Caching and Prefetching Techniques”

ABSTRACT

This work presents a bibliographical revision of Web caching and prefetching techniques. The objective of this revision is to help define a pre-fetching strategy allied with a Web caching system aiming at increasing the availability of systems that operate in disconnected or loosely connected mode. There is great number of research being carried pointing out strategies to increase the scalability of Web services, reduce the network traffic and reduce the load of servers, given that by doing so will provide the users with a much better service. However, regarding disconnected mode operations, Web caching alone is not sufficient if we consider the case where the system is totally disconnected. In this case, it becomes necessary to use prefetching techniques. Prefetching is understood as a technique for retrieving data presumably, making it available for future requests. These techniques reduce dramatically the latency perceived by the user and make the state of the connection transparent. The use of prefetching techniques varies in accordance with the context of the application. In this direction, some techniques found in literature will be presented and discussed in this work, analyzing its advantages and disadvantages.

Keywords: Web caching, prefetching, disconnected mode.

1. INTRODUÇÃO

A *World Wide Web* pode ser considerada como um grande sistema de informações distribuído que provê acesso a objetos de dados compartilhados (WANG, 1999). O constante crescimento da rede em todos os sentidos traz e trará muitos desafios. Atualmente, alguns dos maiores desafios estão relacionados ao congestionamento da rede (aumentando a latência aparente para os usuários) e sobrecarga dos servidores. Neste contexto se insere o conceito de *Web caching*, que tem por finalidade reduzir este tipo de problema através do armazenamento de objetos populares próximo aos clientes. Estes objetos acabam por não precisarem ser requisitados na origem sempre que solicitados por um usuário. O uso adequado destes sistemas é capaz de trazer vários benefícios a todos os agentes envolvidos nos serviços *Web*.

A primeira idéia de *Web caching* surgiu com os servidores de *proxy*, que armazenam as requisições processadas antes de enviá-las aos clientes. Uma vez armazenadas essas cópias poderiam ser passadas sempre que solicitadas novamente, sem necessidade de requisição ao servidor de origem. Esta abordagem foi bem sucedida, pois, servidores de *proxy* normalmente pertencem a redes de organizações, cujos clientes possuem interesses comuns.

Apesar de bem sucedida, algumas outras abordagens começaram a surgir, sendo necessário criar uma classificação de acordo com a localização na rede: *browser cache* (cliente), *proxy cache* (*proxy*) e *proxies reversos* (servidor). Cada uma destas abordagens atende aos objetivos dos sistemas do qual os *caches* fazem parte. Além das possibilidades diferentes de localização do *cache*, existem algumas outras decisões a serem tomadas com relação aos sistemas de *Web caching*. A primeira delas diz respeito ao algoritmo de substituição que deve ser usado. Uma outra é relacionada aos métodos de manutenção de consistência no *cache*.

Além dos problemas relativos à expansão da Internet, um novo problema tem surgido nos últimos tempos. Ele diz respeito ao aumento no número de dispositivos móveis existentes. Esta grande utilização de dispositivos móveis aliados ao crescimento dos pontos de acesso à rede traz mais desafios. O maior destes desafios está relacionado com a ausência de conexão em determinados momentos. Buscando aumentar a disponibilidade ao usuário, técnicas de busca antecipada de informações (*prefetching*, *hoarding*) estão começando a ser aliadas a sistemas de *Web caching*. A utilização de mecanismos de busca antecipada adiciona um outro problema aos pré-existentes: o que e quando buscar antecipadamente?

Portanto, este trabalho tem como objetivo o estudo e uma avaliação qualitativa de técnicas de gerenciamento de *Web caching* e busca antecipada de informação a fim de se propor uma arquitetura que uma as duas técnicas e seja capaz de resolver o problema da operação em modo desconectado de forma eficaz.

O restante deste trabalho está organizado da seguinte maneira. O capítulo 2 apresenta uma introdução a *Web caching*, trazendo suas vantagens, desvantagens e características desejáveis. O capítulo 3 apresenta a classificação dos *Web cachings* segundo a sua localização na rede. A seguir, o capítulo 4 apresenta as maneiras de se organizar os *caches* de forma a aumentar a efetividade dos mesmos. O capítulo 5 traz uma análise dos métodos de substituições encontrados na literatura. O capítulo 6 apresenta estratégias para manutenção da consistência dos objetos em *cache*. O capítulo 7 apresenta a revisão bibliográfica referente à busca antecipada de informações, apresentando sua classificação quanto ao contexto *Web* e quanto aos mecanismos de predição utilizados. Para finalizar, o capítulo 8 traz um sumário e as conclusões relativas a este trabalho.

2. WEB CACHING

Este capítulo apresenta uma introdução ao tema iniciando com um breve histórico de memória *cache*. Serão vistas algumas vantagens e alguns problemas que devem ser tratados ao se desenvolver um sistema de *Web caching*. Em seguida, algumas propriedades desejáveis a um sistema de *Web caching* serão apresentadas. Ao final serão analisadas as principais maneiras de avaliação de performance de um sistema de *Web caching*.

2.1. Histórico

O termo *cache* é de origem francesa e tem como significado “armazenar”. Em computação o termo diz respeito ao armazenamento de algumas informações em um local onde possam ser facilmente acessadas no futuro.

O conceito de *cache* surgiu na computação no sentido de melhorar o desempenho através de cópias de informações em locais que facilitassem seu acesso. Atualmente, o conceito de *caching* pode ser encontrado em quase todas as áreas da computação. Processadores possuem *cache* para diminuir o descompasso com relação à memória principal. Sistemas Operacionais usam *caches* para discos e sistemas de arquivos. Sistemas de arquivos distribuídos apóiam-se fortemente em seus métodos de *caching* para aumentar sua performance. Máquinas de busca na Internet usam *cache* para melhorar desempenho em suas pesquisas.

O bom funcionamento do *caching* se deve ao princípio da localidade de referência, que diz que acessos a informações próximas são prováveis de acontecerem em pequenos intervalos de tempo. Existem dois tipos de localidade de referência: temporal e espacial. Localidade espacial indica que um endereço próximo da referência atual será muito provavelmente acessado num futuro próximo, enquanto localidade temporal significa que dados e instruções utilizados recentemente serão, provavelmente, utilizados

novamente. Quando estas previsões são corretas, é possível notar um incremento significativo na performance do sistema.

É comprovada na prática a eficiência do *caching* em memória e sistemas de arquivo. Com o grande crescimento do tráfego na *Web*, percebeu-se que a estratégia de *caching* seria uma técnica que possivelmente reduziria a latência da rede. Daí surge o termo *Web Caching*, que nada mais é que o armazenamento de cópias de informações de provável acesso num futuro próximo em locais onde um usuário as acesse de forma rápida e fácil. Da mesma forma que *caches* de memória e disco, que mantêm os dados mais acessados em uma área específica para posterior recuperação, o *Web cache* armazena os objetos (páginas HTML, imagens, arquivos, etc) acessados na Internet.

2.2. *Web Caching*: Usar ou Não Usar?

Experiências e medidas realizadas mostram efeitos positivos e negativos da utilização de *Web caching*.

A utilização do *Web caching*, segundo Wessels (2001) se justifica através da frase “tempo é dinheiro”. Esta economia esperada de tempo se dá através de mecanismos que distribuem cópias das informações disponíveis na *Web* em vários locais diferentes. Estes mecanismos deixam a informação mais próxima dos usuários finais, facilitando a localização e diminuindo a latência na recuperação dos dados.

São três as principais vantagens em se fazer *caching* do conteúdo *Web*:

- *a latência entre pedido e resposta é reduzida*, fazendo com que as páginas sejam carregadas mais rapidamente;
- *o consumo de banda de rede é reduzido*, diminuindo assim o tráfego e o congestionamento da rede;
- *a carga no servidor Web de origem é significativamente reduzida*, através da distribuição dos dados entre *proxies* espalhados pela rede.

A primeira das vantagens é a principal e mais citada quando se trata de *caching*. Latência inclui, basicamente, o tempo que um objeto leva para ser transferido do servidor de origem até o *proxy* (latência externa) e o tempo de transferência do objeto do *proxy* até o cliente (latência interna). Quando ocorre um *hit* no *cache* (isto é, o objeto solicitado é encontrado no *cache*), a latência externa é totalmente eliminada. Como pode

ser observada na figura 2.1, a maior perda está entre o *proxy* e o servidor de origem. Esta perda está relacionada ao tempo de solicitação do dado, *overlapping* do servidor e tempo de resposta ao *proxy*. Segundo estudos realizados por Kroeger, Long e Mogul (1997) um *hit* no *cache* pode resultar numa redução na latência entre 77% e 88% se comparado com um sistema que não utiliza *cache*. Simulando um *cache* de tamanho ilimitado chegou-se a uma redução média da latência entre 22% e 26%.

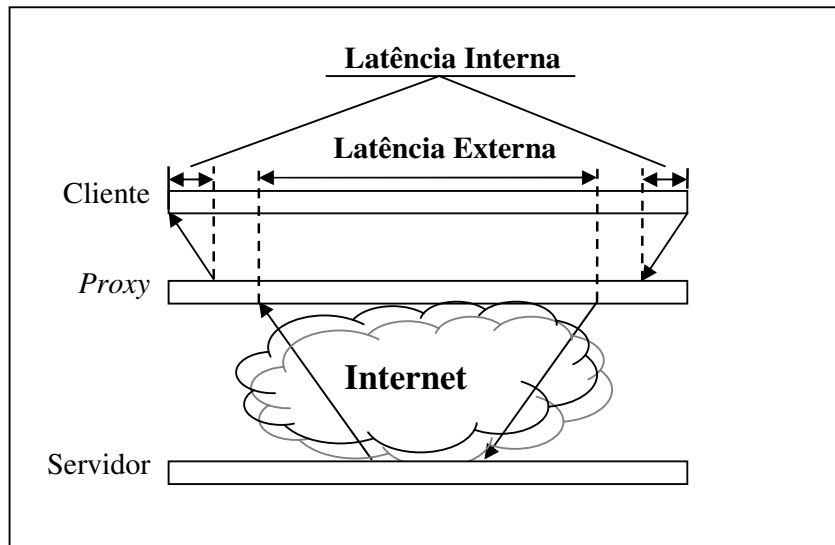


Figura 2.1 - Latência de uma requisição ao servidor de origem

Uma justificativa para a busca por melhoras na latência é que, do ponto de vista dos usuários, um melhor tempo de resposta às suas requisições, aumentam o grau de satisfação. Segundo Krishnamurthy e Rexford (2001) uma fração significativa dos cancelamentos que ocorrem durante uma sessão do usuário normalmente é o resultado de uma frustração do usuário em não obter respostas rapidamente.

A segunda vantagem diz respeito à redução do consumo de banda de rede. Reduzindo o consumo de banda não apenas reduz o custo da rede, como também reduz a utilização do link e do servidor de origem, reduzindo (de certa forma) a latência externa. Um estudo a respeito, diz que 90% do tráfego de clientes de *cable modems* é de responsabilidade do HTTP (ARLITT; FRIEDRICH; JIN, 1999). A utilização de *Web cache* reduz a banda utilizada pelo tráfego HTTP, ocasionando um aumento na performance de outras aplicações.

A terceira vantagem observada se refere à diminuição na carga do servidor *Web*. Ao reduzir o tráfego entre *proxy* e servidor, o número de requisições ao servidor diminui.

Sendo assim, temos uma redução na carga do mesmo, melhorando seu desempenho. Esse melhor desempenho no servidor reflete numa melhora na latência externa, nos casos de *cache miss*, aumentando ainda mais a performance.

Expostas estas vantagens, nota-se um certo ganho por todas as partes envolvidas no acesso à *Web*. Os usuários experimentam uma rede mais rápida, devido à redução da latência na transferência de informações. A rede é favorecida devido à diminuição no desperdício de banda com dados redundantes, deixando largura de banda disponível para outros dados passarem. E é favorável aos ISPs, que experimentam uma diminuição na carga de seus servidores.

Até aqui foi mostrado o que o *cache* traz de bom, mas a utilização traz alguns efeitos colaterais.

Manter a consistência de um Web cache é um problema muito complexo. A princípio, pode parecer simples manter um *cache* atualizado, bastaria perguntar ao servidor sobre a validade dos seus dados. Isto acarretaria muitas requisições ao servidor, voltando a ocasionar problema de latência, aumentando a carga no servidor e a utilização de banda. No caso de aumentar-se o intervalo de tempo entre verificações da validade dos dados, pode-se estar criando o problema de *acesso a informações velhas*.

A latência pode aumentar no caso da ocorrência de muitos *misses*, uma vez que existe o tempo de busca e o tempo de armazenamento do objeto no *cache*.

Cache pode trazer complicações aos servidores, pois causam *distorções nos arquivos de log*. Dados como número de *page views*, locais de acesso, frequência com que certos usuários acessam a página ficam distorcidos e passam a não ser analisados corretamente.

Um único proxy é sempre um gargalo. Deve existir um limite de clientes que um *proxy* pode servir. Este limite deve manter o *proxy* no mínimo tão eficiente quanto se estivesse sendo usada uma conexão sem *proxy*.

A “*cacheabilidade*” dos elementos deve ser considerada sempre. O número de páginas personalizadas e geradas dinamicamente tem crescido e isso pode trazer graves problemas. Uma página personalizada para uma pessoa X pode ser posta em *cache*, e ser mostrada para uma pessoa Y, quando da requisição da mesma. Existem ainda as páginas que dependem de preenchimento de formulário. Estas normalmente são acessadas uma única vez, e apenas ocuparão espaço no *cache*.

Têm-se ainda problemas com privacidade, armazenamento de conteúdo ofensivo, integridade das informações contidas no *proxy*, veracidade das informações (pode-se estar comprando gato por lebre), direitos autorais e propagandas indesejadas.

2.3. Propriedades Desejáveis a um Sistema de *Web Caching*

Segundo Wang (1999) existem algumas propriedades desejáveis aos sistemas de *Web caching*, são elas: rapidez no acesso, robustez, escalabilidade, transparência, eficiência, adaptatividade, estabilidade, balanço de carga e simplicidade. Nesta seção estas propriedades serão discutidas, uma a uma.

2.3.1. Rapidez no Acesso

Como dito na seção anterior, a satisfação do usuário está diretamente ligada a rapidez no acesso às informações. Com outras palavras, a medida de qualidade do usuário é influenciada pela latência do acesso. É necessário que um sistema de *Web caching*, mesmo com o aumento da latência interna, reduza a latência geral dos acessos. Do ponto de vista do usuário, a latência observada, na média, deve ser menor do que se estivesse sendo usada uma conexão sem *proxy*.

2.3.2. Robustez

Sistemas de *Web caching* aumentam a disponibilidade das informações. Disponibilidade é outro ponto que aumenta a satisfação dos usuários. Com o uso de *cache* é possível aumentar a disponibilidade dos dados, estando estes acessíveis a qualquer momento pelos usuários, escondendo possíveis problemas com o servidor de origem, ou com a rede.

Wang (1999) cita três aspectos importantes com relação à robustez de sistemas de *Web caching*: (i) a queda de um *proxy* não pode conduzir a uma queda no sistema; (ii) o sistema de *cache* deve ser tolerante a faltas; e (iii) o sistema de *cache* deve ser desenvolvido de forma que seja fácil recuperá-lo em caso de falhas.

2.3.3. Transparência

É intuitivo dizer que um sistema de *Web caching* deve ser transparente ao usuário, visto que as únicas respostas que lhe interessam são rapidez e maior disponibilidade. Porém existe uma grande discussão com relação a este ponto. Existem situações em que o usuário deve ser informado e inclusive, deve interagir com o sistema para, de certa forma, ajudar o sistema.

De maneira geral, em aplicações usuais, os *caches* devem atuar de modo transparente. Exemplos onde se encontram sistemas de *cache* desse tipo são *proxies* de universidades, de empresas e de ISPs. Em aplicações mais específicas, deve-se deixar o usuário ciente de certas características e acontecimentos.

Em sua abordagem, que diz respeito a *caching* em dispositivos móveis, Dix e Beale (1996) defende que o usuário esteja ciente de sua conectividade. Ele argumenta que transparência é a abordagem errada nesse tipo de sistemas, pois ela esconde informações que podem ser complementadas pelos usuários. Sua proposta é um sistema que deixe os usuários cientes das informações apropriadas, sem que isto atrapalhe a sua tarefa principal. Estas informações podem (ou não) ser passíveis de interatividade com os usuários, podendo o usuário dar dicas ao sistema em alguns casos.

Experiências realizadas com o Coda (EBLING; JOHN; SATYANARAYANAN, 2002) mostraram que usuários experientes freqüentemente testam seu *cache* desconectando-se e tentando acessar informações antes de deixar uma área com rede. Isto mostra que, existem casos em que os usuários desejam saber o conteúdo disponível em *cache*. Vale lembrar que Coda utiliza *cache* no lado cliente para manter dados acessíveis em modo desconectado.

Atualmente, grande parte dos sistemas que lidam com operação em modo desconectado encontrou a necessidade de deixar o estado da conectividade visível aos usuários, dando a eles um maior controle sobre certos aspectos do sistema. A possibilidade de influenciar, interagir e “ensinar” o sistema, são os grandes atrativos da chamada “Translucidez”. O grande problema agora é identificar quais aspectos devem ficar visíveis aos usuários.

2.3.4. Escalabilidade

A *Web* tem crescido muito nos últimos anos, e tende a crescer ainda mais nos próximos anos. Isto impõe a necessidade dos sistemas de *cache* serem escaláveis. É desejável que um sistema de *cache* se adapte, pelo menos, a problemas como crescimento de usuários e densidade da rede, adaptando seu tamanho e o número de replicações.

2.3.5. Eficiência

O principal ponto a se avaliar quanto a eficiência de um sistema de *Web caching* é o *overhead* imposto pelo sistema e o quanto ele aumenta a latência da rede. É necessário

que um sistema de *caching* adicione o mínimo de sobrecarga à rede. Outro ponto que se deve atentar é que o sistema de *caching* não deve subutilizar recursos críticos da rede.

2.3.6. Adaptatividade

Quando se fala em crescimento na *Web*, não se fala apenas de crescimento em número ou área de cobertura. Esse crescimento se dá também com relação à heterogeneidade de usuário e meios de acesso à rede. Tal heterogeneidade pode ser com relação a necessidades e interesses dos usuários, com relação à eficiência de sistemas ou com relação às restrições e limitações das plataformas utilizadas atualmente (PCs, PDAs, Telefones). É desejável, então, que os sistemas de *Web caching* possuam métodos de se adaptar a toda essa heterogeneidade.

Também é necessário que o sistema de *Web caching* possa se adaptar às alterações dinâmicas impostas pela rede e pela demanda de requisições. Esta adaptatividade envolveria desde gerenciamento do *cache* até localização do *proxy*. Por exemplo, de acordo com padrões de conectividade entre o *proxy* e os servidores, o *proxy* poderia buscar rotas alternativas que minimizariam o tempo de resposta.

2.3.7. Estabilidade

Os esquemas utilizados em um sistema de *Web Caching* não podem introduzir instabilidades à rede (WANG, 1999).

2.3.8. Balanceamento de Carga

É de grande importância que um sistema de *caching* distribua a carga, se possível, por toda a rede. Um único *proxy* para muitos clientes, além de se tornar um ponto de falha único, pode acabar se tornando um gargalo e pode acabar piorando o serviço de uma determinada parte da rede.

2.3.9. Simplicidade

Simplicidade sempre deve ser considerada em sistemas de informação. Sistemas simples são mais fáceis de implementar e são mais bem aceitos como padrões internacionais (WANG, 1999).

2.4. Medindo a Performance de sistemas de *Web Caching*

Existem algumas métricas que se relacionam à performance de *Web Caches*. As mais normalmente adotadas são *hit ratio (HR)* e *byte hit ratio (BHR)*. Em alguns casos

se utiliza o tempo de resposta médio como maneira de medir, principalmente para analisar a latência.

Hit Ratio é o percentual de requisições satisfeitas pelos objetos armazenados em *cache*, em relação ao número total de requisições:

$$HR = \frac{hits}{hits + misses}$$

Byte Hit Ratio (BHR) é o percentual de bytes requisitados pelo cliente que foram enviados pelo *cache*, sem solicitação ao servidor de origem. Sendo $h_1, h_2, \dots, h_i$ o tamanho de cada um dos i objetos enviados após um *hit* no *cache*, $m_1, m_2, \dots, m_j$ o tamanho de cada um dos j objetos recuperados do servidor (*miss*) e $i + j$ o número total de requisições ao *cache*, tem-se:

$$BHR = \frac{\sum h_i}{\sum h_i + \sum m_j}.$$

Além destas duas medidas pode-se utilizar ainda o tempo de resposta ao usuário. Esta métrica é importante para demonstrar a performance de um sistema com relação ao tempo de espera do usuário e latência média. O problema é como medi-lo de maneira ideal.

2.5. Considerações Finais

O presente capítulo apresentou um histórico de memória *cache*, mostrando como ela pode ser aplicada à *Web*, não servindo apenas para aumentar a performance, mas também para aumentar a disponibilidade de objetos. Com a união destas duas características temos um sistema que traz respostas mais rápidas ao usuário, e que “esconde” possíveis erros que ocorram com o servidor ou com a rede. Foi visto também que, existem alguns problemas que devem ser tratados ao se desenvolver um sistema de *Web caching*.

Algumas propriedades desejáveis a um sistema de *Web caching* foram apresentadas. A propriedade de transparência mereceu uma atenção especial, pois pesquisadores têm realizado experimentos que mostram que existem casos em que se faz necessária a interação entre usuário e sistema. Esta “translucidez” visa aumentar a satisfação dos usuários e a eficiência do sistema, que se aproveita do conhecimento do usuário.

Por fim foram mostradas as principais métricas utilizadas para analisar a performance de um sistema de *Web caching*.

3. TIPOS DE WEB CACHES

O conteúdo da *Web* pode ser armazenado em vários locais diferentes entre o cliente e o servidor de origem. Muitos *browsers* possuem um *cache* embutido, os chamados *browser caches*. Seguindo a cadeia requisição-resposta, pode-se encontrar os *proxy caches*, que armazenam os objetos de acordo com as requisições de um determinado grupo de clientes. Um tipo especial destes *proxies* não exige configurações por parte do cliente, pois interceptam requisições HTTP, sendo chamados *proxies* de interceptação (ou *proxy cache* transparente). No outro extremo da cadeia existem os *proxies* reversos (ou *surrogates*), responsáveis por armazenar as respostas mais comuns dos servidores. A seguir serão detalhados os tipos de *Web caching*.

3.1. *Browser Cache*

A maioria dos *browsers* conhecidos possui um *cache* embutido. Através deste *cache* muitos arquivos podem ser reutilizados, quando se visita novamente um mesmo site ou quando páginas *Web* utilizam os mesmos logos, figuras, *banners*. Este tipo de *caching* é realizado, pois é comum o acesso a uma mesma página múltiplas vezes em um curto espaço de tempo (por exemplo o uso do botão Voltar do *browser*).

Geralmente os *browsers* permitem que os usuários definam os parâmetros, tais como quanto espaço se deseja reservar para o *cache* e frequência com que as informações do *cache* devem ser invalidadas.

Apesar muito úteis, esses *caches* apresentam alguns problemas. Os dados armazenados são correspondentes às requisições de apenas um usuário. Isto quer dizer que só haverá um *hit* no caso de uma página ser revisitada. Outro problema é a incompatibilidade de *caches* de diferentes *browsers*. Este último problema já é alvo de pesquisas. Existem algumas soluções comerciais que são compatíveis com um grande número de *browsers*.

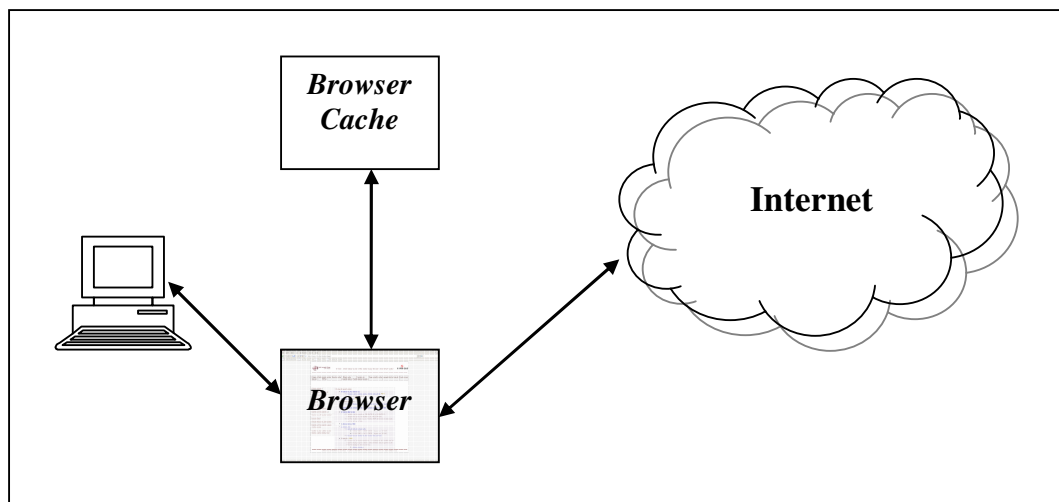


Figura 3.1 – Funcionamento básico de um *Browser Cache*

3.2. *Proxy Cache*

Este tipo de *cache* pode servir a vários usuários de uma só vez. Uma vez acessado e utilizado por muitos clientes, os acessos ocorrerão em maior número e de forma mais diversa. Sendo assim, o *proxy* será mais diversificado, aumentando assim o seu *hit ratio*. Este tipo de *proxy* normalmente apresenta um *hit ratio* mais alto que os *browser caches*.

Ao receber uma requisição, o *proxy cache* procura pelo objeto localmente. Se a encontrar (*hit*), este é prontamente repassado ao usuário. Caso contrário (*miss*), o *proxy* faz uma requisição ao servidor, grava a página no disco e a repassa ao usuário. Requisições subsequentes (de qualquer usuário) recuperam a página que está gravada localmente. Os servidores do tipo *proxy cache* são utilizados por organizações ou provedores que querem reduzir a quantidade de banda do sistema de comunicação que utilizam.

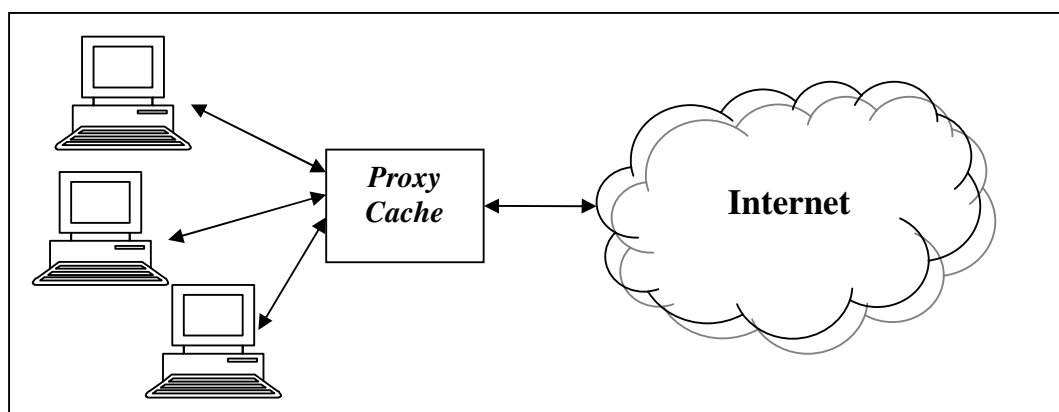


Figura 3.2 – Funcionamento básico de um *Proxy Cache*

3.3. *Proxies de Intercepção (Transparent Proxy Cache)*

Segundo Wessels (2001) um das maiores dificuldades de operação de um *proxy cache* é conseguir usuários para usar o serviço. Isto se deve à dificuldade de se configurar os *browsers*. Usuários podem pensar não estarem configurando seus *browsers* corretamente e, por isso, acabarem por desabilitar o uso do mesmo. Outro problema com relação aos usuários é a possível resistência ao uso de *proxy* devido ao medo de receber informações desatualizadas.

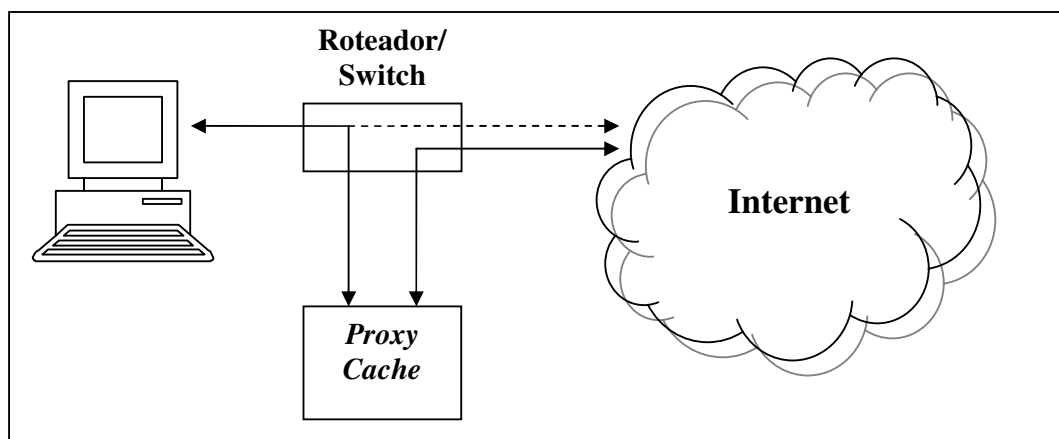


Figura 3.3 – Funcionamento de um *Proxy* de Intercepção

Tendo em vista estes problemas, muitas organizações passaram a usar *proxies* de intercepção, uma vez que eles diminuem a sobrecarga dos administradores e aumentam o número de clientes utilizando o *proxy*. A idéia principal deste tipo de *proxy* é trazer o tráfego para o *cache*, sem a necessidade de configuração dos clientes. Isto é feito através do reconhecimento das requisições HTTP pelos roteadores, e redirecionamento das mesmas ao *proxy*.

3.4. *Proxies Reversos (Surrogates)*

Surgiu da necessidade de aproximar os *proxies* dos servidores para reduzir a carga sobre eles. Recebem este nome, pois estão na ponta contrária ao tradicional na cadeia requisição-resposta.

Estes *proxies* são também chamados de aceleradores, pois o sistema de armazenamento em *cache* fica à frente de um ou mais servidores *Web*, interceptando solicitações e agindo como um *proxy*. Os documentos armazenados em *cache* são fornecidos a uma maior velocidade, enquanto os que não estiverem em *cache* (conteúdo

dinâmico/personalizado) são solicitados aos servidores *Web* de origem quando necessário.

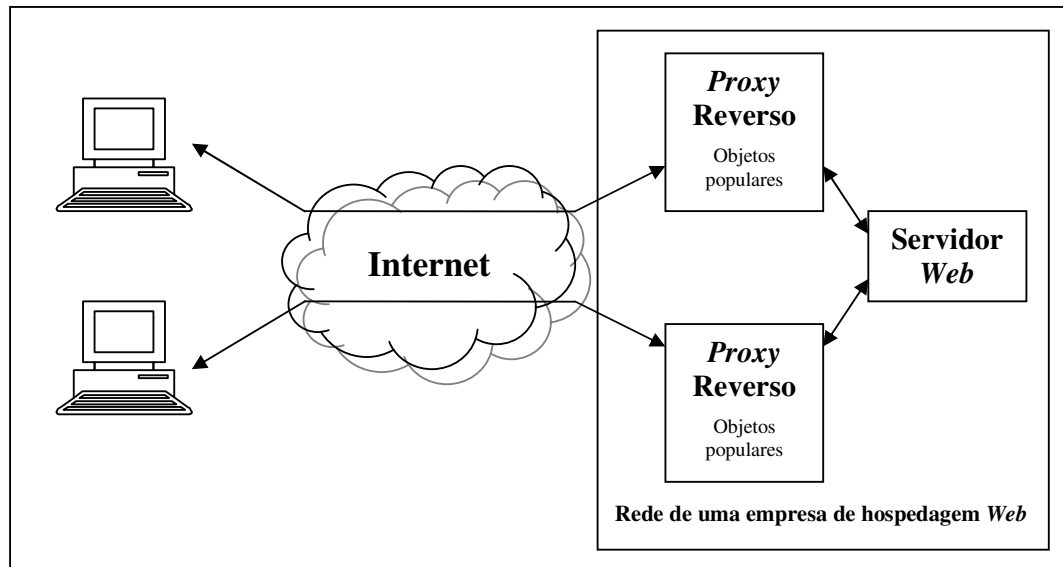


Figura 3.4 – Funcionamento de um *Proxy Reverso*

3.5. Considerações Finais

Este capítulo traz uma revisão sobre os tipos de sistemas de *Web caching* que podem ser encontrados e seu funcionamento. O possível relacionamento entre tipos diferentes de *Web caches* pode trazer grandes vantagens, aumentando a satisfação do usuário. No próximo capítulo serão mostradas as arquiteturas utilizadas de forma a haver uma cooperação entre *caches*, buscando aumentar o número de *hits*.

4. ARQUITETURAS PARA *WEB CACHING*

A eficiência de um sistema de *Web caching* depende do número de clientes atendidos por ele. Quanto maior o número de clientes, maior a probabilidade de se encontrar um documento armazenado em *cache*. Os *caches* podem cooperar entre si, trocando informações quando necessário, aumentando assim a probabilidade de encontrar uma informação sem solicitar ao servidor de origem. Uma arquitetura de um sistema de *Web caching* deve fornecer meios eficientes de comunicação inter-*caches*, aumentando este grau de cooperação, aumentando assim a sua performance.

4.1. Arquitetura Hierárquica

Uma maneira de organizar os *caches* para haver cooperação é através de uma hierarquia. Nesta hierarquia, cada um dos *caches* é colocado em um diferente nível. Numa hierarquia simplificada, consideram-se quatro níveis: clientes, *caches* institucionais, *caches* regionais e *caches* nacionais (RODRIGUEZ; SPANNER; BIRSACK, 2001), como pode ser observado na figura 4.1. Esta arquitetura é atrativa, pois oferece aumentos notórios na performance, reduzindo o uso de banda de rede e aumentando as velocidades de *download*.

Quando um cliente faz uma requisição de um objeto este é buscado primeiramente no *browser cache*. Não sendo encontrado o objeto requisitado a requisição é passada ao *cache* institucional. Caso o objeto ainda não seja encontrado, a mesma requisição é passada ao *proxy* regional, que repassa as requisições não satisfeitas ao nível nacional de *cache*. Se o objeto não for encontrado em nenhum nível, a requisição é mandada ao servidor de origem. Quando o objeto é encontrado em algum nível (ou no servidor de origem) este trafega toda a hierarquia, deixando uma cópia em cada um dos *caches* intermediários.

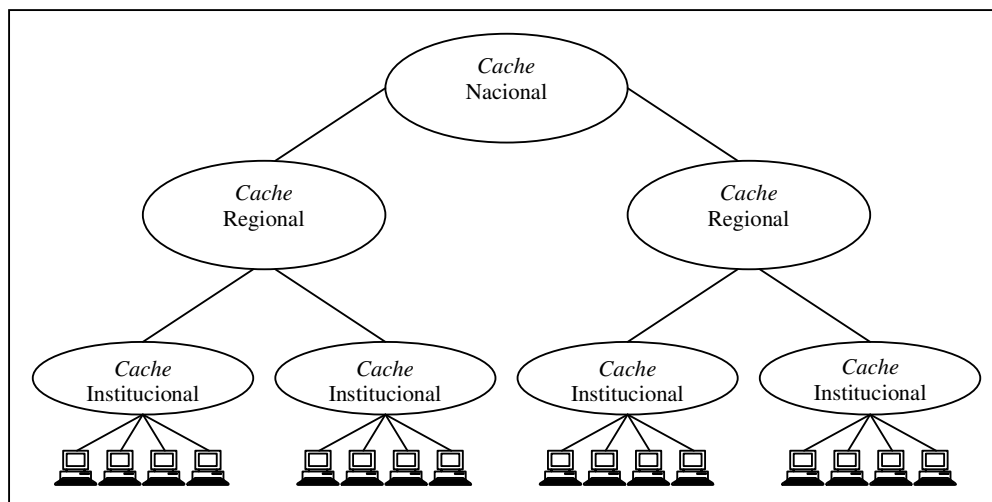


Figura 4.1 – Estrutura hierárquica, mostrando o posicionamento dos *caches*.

Esta arquitetura foi primeiramente apresentada no projeto *Harvest* (CHANKHUNTHOD et al., 1996), com o intuito de reduzir mais ainda a demanda de uso de largura de banda e a carga nos servidores de origem. Foi criada, então, uma estrutura hierárquica de forma que os *misses* de um *cache* fossem atendidos por outros, diminuindo os acessos ao servidor. O projeto *Harvest* não está mais ativo, mas é de enorme significância na área, pois deixou como parte de seus resultados o *proxy Squid* (WESSELS; CLAFFY, 1998).

Apesar de apresentar incremento na performance, esta arquitetura apresenta alguns problemas, a saber:

- cada nível da hierarquia insere um atraso adicional à latência total;
- os *caches* nos níveis de hierarquias mais altos são potenciais gargalos e podendo ocasionar grandes atrasos;
- *caches* dos níveis mais altos precisam ter espaços de armazenamento muito grandes; e
- múltipla cópias de um mesmo documento replicado em todos os níveis da hierarquia.

4.2. Arquitetura Distribuída

Para tentar resolver os problemas relacionados aos níveis mais altos da arquitetura hierárquica, alguns pesquisadores propuseram uma arquitetura totalmente distribuída. Nesta arquitetura não existem *proxies* intermediários, existindo apenas *proxies*

institucionais junto aos clientes. Estes cooperam entre si para que uns possam atender aos *misses* dos outros. Isto acaba com a necessidade de *caches* de alto nível, que armazenam tudo o que os *caches* inferiores a ele possuem, necessitando grande espaço de armazenamento.

Uma vez que não há mais um *cache* que centraliza todos os documentos requisitados pelos outros níveis, os *caches* institucionais precisam de outros mecanismos para compartilhar seus documentos. Para decidir para onde enviar uma requisição no caso de um *miss*, cada *cache* institucional guarda meta dados sobre o conteúdo de outros *caches* institucionais. Estes dados são trocados freqüentemente entre os *caches*. Em (POVEY; HARRISSON, 1997) é proposta a existência de uma estrutura hierárquica, usada somente para a troca de meta dados.

Existem muitas soluções baseadas na arquitetura distribuída implementadas e sendo utilizadas atualmente. Uma abordagem é apresentada pelo CARP (*Cache Array Routing Protocol*) (CARP, 2004), que usa uma função *hash* para decidir para qual *cache* a requisição será repassada. Povey e Harrison (POVEY; HARRISSON, 1997) propuseram um *cache* distribuído em que os *caches* de nível superior são trocados por servidores de diretórios que contém a dicas sobre a localização dos documentos armazenados em cada *cache*. Como já dito, a estrutura hierárquica é usada apenas para facilitar a distribuição dos metadados referentes à localização dos documentos.

4.3. Arquitetura Híbrida

Uma arquitetura híbrida busca unir as vantagens das arquiteturas hierárquica e distribuída. Nessa arquitetura os *caches* cooperam em qualquer nível de uma hierarquia usando *caching* distribuído. Experiências realizadas por Rodriguez, Spanner e Biersack (2001) mostraram que um esquema híbrido com *caches* cooperando em todos os níveis incrementa a performance reduzindo a latência, a largura de banda utilizada e a carga nos *caches*.

O grupo Harvest desenvolveu o ICP (*Internet Cache Protocol*) (WESSELS; CLAFFY, 1998) que apóia a recuperação de documentos tanto em *caches* vizinhos como em *caches* em níveis de hierarquia superior, trazendo o documento do *proxy* que possuir o menor RTT (*Round Trip Time*).

4.4. Considerações Finais

A pesquisa realizada por Rodriguez, Spanner e Biersack (2001) mostra que a estrutura hierárquica de *caches* apresenta tempos de conexão menores do que a estrutura distribuída através do posicionamento de cópias de documentos em níveis intermediários próximo a clientes. A pesquisa mostra ainda que *caching* distribuído tem tempos de transmissão menores que usando estrutura hierárquica, apesar de utilizar mais largura de banda. Por último é mostrado que um esquema híbrido bem configurado pode combinar as vantagens das estruturas hierárquica e distribuída, reduzindo tanto o tempo de conexão como o tempo de transmissão.

5. POLÍTICAS DE SUBSTITUIÇÃO

Uma questão muito importante com relação a *Web caching* é determinar quando os objetos serão atualizados ou removidos, de modo a garantir a sua consistência. Muitos algoritmos de substituição de *cache* tem sido propostos em estudos recentes, tentando minimizar algumas métricas, como hit rate, byte hit rate e tempo de resposta médio.

Neste capítulo serão apresentadas algumas das estratégias de substituição utilizadas em *Web Caching*. Estas estratégias estão divididas, conforme proposto por Podlipnig e Böszörményi (2003), em cinco diferentes classes, a saber: (i) estratégias baseadas nos mais recentes; (ii) estratégias baseadas nos mais frequentes; (iii) estratégias baseadas nos mais recentes/frequentes; (iv) estratégias baseadas em função; e (v) estratégias randômicas.

5.1. Estratégias Baseadas nos Mais Recentes

A grande maioria das técnicas de substituição utiliza este tipo de estratégia, sendo a quase totalidade variação do conhecido LRU (*Least Recently Used*). A seguir serão apresentadas algumas destas estratégias.

5.1.1. LRU (*Least Recently Used*)

É a política de substituição mais popular existente. Como o próprio nome diz esta estratégia remove os elementos que não são usados há mais tempo. É uma estratégia muito simples, pois não requer parametrização. É de grande eficiência quando há grande localidade temporal em sua carga, em outras palavras, quando os objetos mais recentes são mais prováveis de serem acessados num futuro próximo. Sua grande desvantagem dentro da *Web* é não levar em consideração o tamanho dos objetos, ou o tempo necessário para carregá-lo.

5.1.2. LRU-threshold

É uma variante do algoritmo LRU. O LRU-Threshold (ABRAMS et al., 1995) previne a situação em que um documento grande (se comparado ao tamanho do *cache*), causa a substituição de muitos documentos menores. Isto é feito através de uma restrição que impõe um limite de tamanho aos objetos a serem armazenados, apenas documentos menores que este limite (*threshold*) podem entrar no *cache*.

5.1.3. LRU-Min

Outra variante do LRU, o LRU-Min (ABRAMS et al., 1995) tenta minimizar o número de documentos substituídos. Considere L e I como uma lista e um inteiro, respectivamente. 1. Atribua a I o tamanho do documento solicitado; 2. Atribui a L todos os objetos de tamanho menor ou igual a I ; 3. Remova os LRU da lista até a lista esvaziar ou até o espaço disponível em *cache* ser maior ou igual a I ; 4. Se o espaço livre não for pelo menos I , atribua $I/2$ a I e volte ao passo 2.

5.1.4. SIZE

O algoritmo SIZE (WILLIAMS et al., 1996) usa o tamanho do objeto como primeiro critério de remoção. O objeto de maior tamanho é removido primeiro. No caso de objetos com o mesmo tamanho, se aplica LRU. Da mesma maneira que LRU, essa política é simples e não necessita de parametrização. Um problema desta abordagem é que objetos pequenos podem ficar por muito tempo no *cache*, mesmo sem serem referenciados. Para solucionar este problema costuma-se utilizar um mecanismo de expiração em paralelo.

5.1.5. Pitkow/Recker

A estratégia de Pitkow/Recker (PITKOW; RECKER, 1994) utiliza LRU para remover os objetos da *cache*, exceto quando todos os objetos foram acessados em um mesmo dia, neste caso o objeto com maior tamanho é apagado.

5.1.6. HLRU (*History-LRU*)

HLRU (VAKALI, 2000) introduz um esquema que controla o histórico do número de referências a um determinado objeto. Tome $r_1, r_2, \dots, r_n$ como requisições à *cache* em tempos $t_1, t_2, \dots, t_n$. A função histórica é definida assim:

$$\text{hist}(x,h) \begin{cases} t_i, & \text{se houverem exatamente } h-1 \text{ referências entre } t_i \text{ e } t_n, \\ 0, & \text{caso contrário} \end{cases}$$

$\text{hist}(x,h)$ define o intervalo de tempo em que ocorreram as h últimas referências a um elemento x em *cache*. Na função, t_i identifica o primeiro dos últimos h acessos ao objeto x . HLRU substitui o objeto com o maior valor para hist . Caso houver muitos objetos com $\text{hist} = 0$ em *cache*, o algoritmo LRU é aplicado.

5.2. Estratégias Baseadas em Frequência

Estas estratégias usam frequência como fator principal. As estratégias existentes são basicamente extensões da LFU conhecida. Elas estão baseadas no fato de que diferentes objetos possuem diferentes popularidades, devido às diferentes frequências de acesso a eles. A seguir serão apresentadas algumas destas estratégias.

5.2.1. LFU (*Least Frequently Used*)

Esta política se baseia num contador de referências para cada objeto. O objeto que possui o menor número de referências (o menos frequentemente acessado) é selecionado para ser substituído. Com relação ao contador a LFU pode ser de dois tipos:

- *Perfect LFU*: Todos os acessos a um objeto são contados. Os contadores persistem à substituição, assegurando que o algoritmo levará em consideração todos os acessos do passado;
- *In Cache LFU*: O contador é inicializado quando do armazenamento do objeto, sem levar em consideração o passado do mesmo.

Normalmente se utiliza a *in cache LFU*, devido à simplicidade de gerenciamento.

5.2.2. LFU-Aging

Com LFU, objetos que foram muito populares num período no passado e não são acessados há muito tempo podem permanecer em *cache*. Para resolver este problema (ARLITT et al. 2000) propuseram a LFU-Aging, que insere uma propriedade de envelhecimento ao LFU. Para tal são inseridos dois parâmetros ao LFU. O primeiro limita o valor médio das frequências. Se o valor médio das frequências de todos os objetos em *cache* ultrapassar este limite, todos os contadores são divididos por 2. O segundo parâmetro limita a frequência individual dos objetos.

5.2.3. LFU-DA (LFU-Dynamic Aging)

A eficiência da LFU-Aging depende dos valores passados como parâmetros para seus limites. LFU-DA (ARLITT et al., 2000) tenta resolver esse problema. Cada requisição ao objeto i dispara uma função que recalcula seu valor K_i através da fórmula $K_i = f_i + L$, onde f_i é a frequência de i e L é um fator de envelhecimento. LFU-DA substitui o objeto com o menor K_i e atribui seu valor a L .

5.3. Estratégias Baseadas nos Mais Recentes/Freqüentes

Nesta seção serão mostradas algumas técnicas que utilizam como base os mais recentemente e freqüentemente usados, além de, às vezes, mais alguns fatores que influenciem na escolha do objeto a ser substituído.

5.3.1. SLRU (Segmented LRU)

A SLRU considera tanto a frequência quanto quão recentemente um objeto foi requisitado para realizar a substituição. Essa política divide o *cache* em dois segmentos: um protegido (para armazenar objetos que são acessados com mais frequência) e outro não-protégido. Quando um objeto é requisitado pela primeira vez, o mesmo é adicionado no segmento não protegido. Quando ocorre um hit no *cache*, o mesmo é movido para o segmento protegido. Os dois segmentos são gerenciados pela política LRU. No entanto, somente os objetos que estão no segmento não-protégido podem ser eleitos para uma substituição. Quando for necessário espaço para adicionar novos objetos, aqueles que foram menos usados no segmento não-protégido são removidos. Objetos removidos do segmento protegido são adicionados no segmento não-protégido como os que tiveram acesso mais recente. Essa política requer um parâmetro, que determina a porcentagem do *cache* que é alocada para o segmento protegido.

5.3.2. LRU*

Na LRU* (CHANG; MCGREGOR; HOLMES, 1999) todos os objetos requisitados são armazenados em uma lista LRU. Cada objeto tem um contador de referências. Quando ocorre um *hit* de um documento, este é movido para o início da lista e seu contador é incrementado de 1. Quando não há espaço suficiente para o objeto solicitado, o contador de *hits* do último elemento da lista é verificado. Se seu valor for zero, o documento é descartado. Caso contrário, o contador de *hits* é decrementado de 1, e o documento é movido para o topo da lista. Um documento apenas é removido no momento em que está no fim da lista e seu contador de *hits* estiver com valor zero.

5.3.3. LRU-Hot

LRU-Hot (MENAUD; ISSARNY; BANATRE, 2000) gerencia duas listas LRU, uma para os hot-objects (populares) e uma para os cold objects (não populares). Um objeto é dito quente, se sua frequência de solicitações no servidor de origem está acima de um limiar. De acordo com esta informação, o objeto é inserido na lista correspondente.

LRU-Hot usa dois contadores: um contador base e um contador para objetos quentes (*hot*). Cada requisição incrementa o contador base em 1. O contador quente (*hot counter*) é incrementado em 1 a cada a ($a > 1$) requisições. Quando um objeto é solicitado ele é armazenado no início de sua lista correspondente e lhe é atribuído um valor de acesso igual ao valor do contador base atual.

No caso de necessidade de substituição, o *cache* recalcula os valores de acesso para os dois objetos que estão no final das duas listas (*tail_hot* e *tail_cold*). O cálculo é feito da seguinte forma:

$$hot_value = tail_hot - hot\ counter$$

$$cold_value = tail_cold - \text{contador base}$$

Então, o objeto que é removido do *cache* é aquele que tiver o menor valor de acesso. Se o *cache* continuar precisando de espaço, o algoritmo recalcula os valores dos próximos dois objetos até que o espaço necessário seja liberado.

5.3.4. HYPER-G

A estratégia Hyper-G (WILLIAMS et al., 1996) combina as estratégias LRU, LFU e SIZE, vistas anteriormente. Primeiramente, é feita a escolha do objeto menos frequentemente usado (*LFU*). Se houver mais de um objeto que satisfaça este critério, o *cache* escolhe o menos recentemente usado (*LRU*). Caso continuem existindo vários objetos nesta situação, escolhe o maior deles.

5.4. Estratégias Baseadas em Função

Estas estratégias utilizam funções matemáticas para calcular o valor de um objeto. Geralmente os autores destas estratégias têm um foco na diminuição de alguma métrica específica ou definem parâmetros para que a função possa se ajustar conforme

necessário. Existem muitas soluções que se utilizam deste tipo de estratégia. Nesta seção serão apresentadas apenas algumas delas.

5.4.1. GD-Size (*GreedyDual - Size*)

GD-Size (CAO; IRANI, 1997) considera o tamanho dos objetos, um custo e o quão recentemente cada um deles foi requisitado para fazer a decisão de substituição. Os objetos são ordenados conforme um valor H definido por $H = \text{custo}/\text{tamanho}$. O objeto que tiver o menor valor de H é selecionado para substituição. Quando o objeto é acessado, o valor de H é calculado e somado com o valor anterior, privilegiando os objetos acessados mais recentes. Essa política não considera o tempo decorrido para recuperar o objeto e a frequência que o objeto foi referenciado.

5.4.2. Bolot/Hoschka

A estratégia de Bolot/Hoschka (BOLOT; HOSCHKA, 1996) usa a seguinte função para calcular o valor do objeto i:

$$f(i) = W_1 l_i + W_2 s_i + \frac{W_3 + W_4 s_i}{T_i},$$

onde s_i é o tamanho do objeto solicitado, l_i é a latência para se recuperar o objeto, T_i é o tempo decorrido desde a última solicitação ao objeto i e W_1 , W_2 , W_3 e W_4 são parâmetros de ajuste. Seus valores dependem da métrica que se deseja maximizar.

5.4.3. HYBRID

Esta proposta de (WOOSTER; ABRAMS, 1997) traz uma função para cálculo do valor de um objeto i no servidor s dependente dos parâmetros: c_s (tempo para contatar o servidor s); b_s (largura de banda disponível para o servidor s). A função é definida como:

$$f(i) = \frac{\left(c_s + \frac{W_b}{b_s} \right) f_i^{W_n}}{s_i},$$

onde W_b e W_n são pesos, f_i é o número de referências feitas ao objeto i e s_i é o tamanho do objeto. Estimativas para c_s e b_s são baseadas no tempo para carregar os documentos do servidor s num passado recente.

5.5. Estratégias Randômicas

Estas estratégias, como o nome diz, usam estratégias randômicas para decidir quais objetos serão substituídos. Nesta seção serão apresentadas, a título de conhecimento da estratégia, duas soluções simples encontradas na literatura.

5.5.1. Rand

Esta estratégia escolhe aleatoriamente um elemento no *cache* e o remove. Caso o espaço não seja suficiente repete a operação.

5.5.2. Harmonic

Esta estratégia é similar à Rand. Harmonic utiliza probabilidades diferenciadas para cada elemento. Esta probabilidade é inversamente proporcional à função de custo do objeto:

$$custo = \frac{c_i}{s_i},$$

onde s_i é o tamanho do objeto e c_i é o custo para trazer o objeto do servidor.

5.6. Considerações Finais

Neste capítulo foi analisado o funcionamento de várias estratégias de substituição para *Web caches* encontradas na literatura. Como pode ser visto cada estratégia visa contornar algum ou alguns problemas, e atacar determinadas métricas. A decisão da melhor política a ser utilizada é uma tarefa não trivial, sendo necessário traçar detalhadamente os objetivos do sistema que está se desenvolvendo. Ficou claro que, em certos casos, se faz necessária a criação de uma estratégia específica para o sistema unindo várias estratégias diferentes ou de uma função que ordene os objetos de forma ideal.

A Tabela 5.1 abaixo traz algumas vantagens e desvantagens das classes de estratégias aqui apresentadas. Estas vantagens e desvantagens são oriundas de uma análise inicial de cada classe.

Classe	Vantagens	Desvantagens
Baseadas nos Mais Recentes	• Consideram a localidade temporal; • são de simples implementação; • são rápidas.	• Não trata tamanho dos objetos de maneira fácil; • não trata frequência (popularidade).
Baseadas em Frequência	• Consideram a frequência (popularidade) dos objetos.	• Complexidade; • poluição do <i>cache</i>; • muitos objetos com valores de frequência iguais.
Baseadas nos Mais Recentes/Freqüentes	• Combinam recência e frequência. Podem prevenir problemas inerentes às estratégias anteriores;	• Estratégias com alta complexidade.
Baseadas em Função	• Possibilidade de otimizar qualquer métrica através dos pesos.	• A escolha dos pesos ótimos é uma tarefa longe do trivial; • determinar a latência para executar cálculos pode introduzir imprecisões.
Randômicas	• Não necessitam estruturas de dados especiais; • são muito simples de implementar	• Impossibilidade de avaliar quantitativamente.

Tabela 5.1 – Vantagens e Desvantagens das estratégias de substituição da *cache*

6. CONSISTÊNCIA DOS OBJETOS

Como dito, *cache* provê baixa latência, porém pode apresentar páginas desatualizadas. Para que o armazenamento em *cache* seja efetivo, não pode haver inconsistências. Uma solução intuitiva seria checar todas as requisições para verificar se o objeto foi atualizado (ou não) na origem, o que seria muito caro, pois ocasiona um grande aumento na latência, no uso da banda e na carga do servidor. Todo o sistema de *cache* deve possuir um mecanismo de verificação da consistência de seus objetos. O problema consiste na implementação de um mecanismo que traga perdas mínimas de performance, o que é muito mais complexo no contexto da *Web*. Neste capítulo serão apresentadas técnicas que buscam resolver o problema de maneira eficiente.

6.1. Comandos HTTP que auxiliam na consistência dos objetos

Wang (1999) apresenta um *overview* dos comandos providos pelo HTTP e que auxiliam os *proxies* na tarefa de manutenção da coerência dos *caches*, estes são apresentados na tabela abaixo:

<i>HTTP GET</i>	Requisita um documento dada uma URL
<i>GET Condicional</i>	HTTP <i>GET</i> combinado com o cabeçalho <i>If-Modified-Since:data</i> pode ser usado por <i>proxies</i> para requisitar uma cópia do documento apenas se este tenha sido modificado.
<i>Pragma: no-cache</i>	Anexado ao <i>GET</i> , este cabeçalho pode indicar que o objeto deve ser carregado do servidor todas as vezes. A maioria dos <i>browsers</i> tem o botão Atualizar, que usa este cabeçalho para solicitar a cópia original.
<i>Last-Modified:data</i>	Retornado junto de todo o <i>GET</i> , indica a última data que a página foi modificada.
<i>Date:data</i>	Indica a última vez que o objeto foi considerado “fresco”, diferentemente do <i>Last-Modified</i> .

Tabela 6.1 – Comandos HTTP que auxiliam nos mecanismos de consistência

6.2. Mecanismos de Consistência do *Cache*

Mecanismos de consistência tentam garantir que as cópias em *cache* estão atualizadas, refletindo o que acontece no servidor. Existem vários mecanismos de consistência atualmente, nesta seção serão apresentados alguns.

6.2.1. Client-validation

É também conhecida como *polling* do cliente. Esta técnica consiste em constantes verificações de inconsistência dos objetos por parte do cliente (*cache*). O *cache* admite que todos os objetos armazenados estão potencialmente desatualizados. O cliente então manda fazer solicitações periódicas ao servidor utilizando *GET condicional (if-modified-since)*. Este mecanismo pode levar a muitas respostas “*Not modified*” por parte do servidor, se o documento não foi alterado. Esta abordagem apresenta a desvantagem de gerar muitas requisições, aumentando o tráfego da rede e a carga do servidor.

6.2.2. TTL (*Time-to-Live*)

TTL se trata de estimativa do tempo de vida de um objeto, determinando o tempo que um objeto pode permanecer em *cache* sem estar desatualizado. Se um objeto requisitado ainda está válido no *cache*, este último o fornece sem contatar o servidor. Caso o objeto esteja inválido (TTL expirado), faz-se uma requisição condicional ao servidor (*if-modified-since*) para verificar se o objeto foi alterado. O servidor retorna um código de status informando se o objeto foi ou não modificado e, no primeiro caso, retorna o novo objeto. TTLs são de simples implementação no protocolo HTTP, bastando usar o cabeçalho *expires*.

6.2.3. Alex Protocol (*TTL Adaptativa*)

A TTL-Adaptativa (CATE, 1992) tenta resolver o problema da geração de muitas requisições ao servidor ajustando os TTLs dos documentos, baseado em observações do tempo de vida. Ele assume, basicamente, que arquivos mais novos mudam mais frequentemente que arquivos antigos. Se um arquivo está há muito tempo em *cache* sem modificações, ele tende a ser estático. O algoritmo utiliza um protocolo que se baseia em um limite de atualização. Este limite é expresso como uma porcentagem da idade atual do documento (data atual – data da última modificação). Um objeto se torna inválido quando sua idade exceder o limite de atualização.

A maioria dos servidores de *proxy* atuais utilizam este mecanismo. O *cache* do Harvest usa esta como sua principal abordagem para manter a consistência, com o limite ajustado para 50. Porém, existem alguns problemas relacionados aos mecanismos baseados em tempo de vida. Primeiro, o *cache* pode retornar dados inválidos se o objeto tiver sido alterado dentro do tempo que a cópia é considerada válida. Segundo, o mecanismo não dá garantias sobre a validade do documento. Por último, os usuários não têm como ajustar o nível de “velhice” tolerável.

6.2.4. Piggyback Client Validation (PCV)

Esta abordagem foi proposta por Krishnamurthy e Wills (1997) e é uma tentativa de reduzir o número de mensagens aos servidores. Um cliente se baseia na utilização da técnica de *Piggyback* para enviar múltiplas requisições de validação em uma mesma requisição HTTP.

Quando um *proxy* precisa enviar uma requisição, ele checa se existe algum outro objeto do mesmo servidor, e verifica se eles expiraram ou estão próximo da data de expirar. Informações sobre estes objetos são enviadas junto da requisição ao servidor. O servidor de origem envia ao *proxy* uma lista de respostas na mesma ordem que elas apareciam na requisição. Um problema desta abordagem diz respeito à frequência de requisições realizadas aos servidores, pois alguns dados podem envelhecer no *cache* se não houver mais requisições a objetos daquele servidor.

6.2.5. Server-Invalidation

O protocolo de invalidação pelo servidor é capaz de garantir uma consistência mais precisa, pois o servidor é responsável por enviar mensagens aos clientes (*caches*) sempre que houver alguma modificação em seus objetos. Este mecanismo, porém, possui alto custo, uma vez que o servidor deve possuir uma lista dos clientes que armazenam cópias de seus objetos para invalidá-las. Outro ponto negativo para este mecanismo é a possibilidade da própria lista se tornar desatualizada, fazendo com que o servidor envie mensagens a clientes que não mais armazenam seus objetos. Existe ainda a possibilidade de haverem clientes indisponíveis, sendo necessário que o servidor continue tentando notificá-los, uma vez que os *caches* só poderão invalidar os objetos se forem notificados pelo servidor.

6.2.6. Piggyback Server Invalidation (PSI)

Krishnamurthy e Wills (1998) propuseram outro mecanismo de invalidação através de piggyback a fim de aumentar a efetividade na coerência do *cache*. Nesta abordagem o servidor de origem designa versões a seus objetos. O servidor incrementa o número da versão a cada vez que um objeto é modificado. Quando o cliente faz alguma requisição ao servidor, é adicionado (*piggyback*) o número da versão do último objeto requisitado e armazenado pelo *cache*. O servidor adiciona (*piggyback*) à resposta a lista de todos os objetos que foram alterados desde o último acesso deste cliente. Os clientes invalidam os objetos da lista que estiverem em *cache*. Como os clientes enviam a informação de versão em suas requisições, o servidor não necessita armazenar a lista de clientes.

6.3. Considerações Finais

No presente capítulo foram apresentados alguns mecanismos que tentam resolver, de forma eficiente, o problema da consistência dos objetos em *cache*. É notável que as estratégias que utilizam *piggyback* apresentam melhores resultados que as outras com relação ao tráfego da rede. Porém, apresentam problemas, como o gerenciamento de números versões de vários servidores (PSI) e falta de periodicidade na requisição de dados de um determinado servidor (PCV). Os outros mecanismos, além de simples, apresentam bons resultados, apesar de aumentarem o tráfego da rede. Possivelmente alguma forma de unir as vantagens destes mecanismos possa favorecer alguma métrica.

7. BUSCA ANTECIPADA DE INFORMAÇÃO

O crescimento da *Web* tem aumentado cada vez mais o número de estudos afim de que seja possível oferecer serviços mais eficientes. Sistemas de *Web caching* são alvo de um grande número de pesquisas, por permitirem melhor desempenho e escalabilidade. Porém, têm-se observado que o *hit ratio* está constantemente caindo, devido ao número de documentos e o tamanho destes documentos estarem crescendo mais rápido que a tecnologia dos *caches* (EL-SADDIK; GRIMODZ; STEINMETZ, 1998).

Uma alternativa para reduzir o atraso sentido pelo usuário é o *prefetching*, ou busca antecipada de informação. A maioria dos usuários navega seguindo *hyperlinks* de uma página para outra. Enquanto os usuários estão lendo a página corrente, existe um tempo em que a rede fica ociosa. Como o usuário geralmente está navegando com um tópico em mente, o tempo de espera da rede pode ser usado para uma busca antecipada de páginas que provavelmente serão acessadas depois daquela. As técnicas de busca antecipada de informação não reduzem a latência, elas apenas utilizam o tempo em que a rede não está sendo utilizada. Isto apenas reduz o tempo de resposta sentido pelo usuário.

Apesar dos benefícios trazidos pelas técnicas de busca antecipada, encontram-se na literatura vários efeitos colaterais possíveis quando as utilizando. O problema mais citado diz respeito justamente ao tráfego adicional causado (FAN; CAO; JACOBSON, 1999; DAVIDSON, 2001), já que a busca pode estar trazendo objetos que podem nem ser utilizados pelo usuário. Outro problema é decidir o que e quando fazer esta busca. As estratégias de busca devem ser modeladas de acordo com a necessidade do domínio sobre o qual estas serão utilizadas.

Neste capítulo serão apresentadas as técnicas de busca antecipada de informação de acordo com o contexto *Web* que elas se encontram (cliente, servidor ou *proxy*). Em seguida serão apresentadas algumas das técnicas de predição utilizadas por mecanismos

de *prefetching*. Por ultimo serão apresentados algumas métricas e trabalhos relacionados.

7.1. Classificação de acordo com o contexto Web

Entendendo-se como servidor *Web* o detentor do objeto original solicitado pelo usuário, e *proxy* como um *cache* coletivo para diversos clientes (*browsers*), ou seja, um ponto intermediário entre estes e os servidores *Web*, pode-se caracterizar a aplicação de pré-busca de três formas distintas no contexto da Internet.

7.1.1. Busca Antecipada Baseada no Cliente

Na busca antecipada do lado cliente, o cliente determina quais paginas devem ser trazidas para o *cache* e requisitá-las. Em (PADMANABHAN; MOGUL, 1996) é apresentada uma análise da redução na latência e no tráfego da rede utilizando *prefetching*. Um problema apresentado por este tipo de abordagem é que se fazem necessárias modificações no *browser* do cliente ou o uso de um *plug-in*. Além disso, sistemas mal planejados podem até dobrar o uso da rede, resultando numa performance pior que sem o uso de busca antecipada. Por último, manter a consistência num *cache* do cliente é extremamente caro. Como resultado, pode ocorrer um aumento na latência da rede e da carga no servidor, tratando dados que podem não ser utilizados pelo cliente.

Cunha e Jaccoud (1997) utilizaram *traces* de clientes *Web* para estudar como a previsão de futuros acessos poderia ser feita. Foi mostrado que alguns modelos funcionam de forma a incrementar a performance de sistemas de *Web caching*. Outras pesquisas propõem meios de minimizar o efeito negativo causado pelas técnicas de busca antecipada através do controle da taxa de transmissão, ou da largura de banda utilizada para fazer tais buscas. Em (FAN; CAO; JACOBSON, 1999) é encontrada uma abordagem que tenta diminuir o problema de latência através de uma técnica de busca antecipada entre cliente e *proxy*. Em sua abordagem Fan utiliza uma técnica de predição baseada na PPM (Prediction by Partial Matching), fazendo *pull* e *push* de informações. Em suas experiências, ele combinava valores diferentes tamanho de *cache* e uso de *delta-compression*, conseguindo redução de cerca de 23% na latência.

7.1.2. Busca Antecipada Baseada no Proxy

Uma busca antecipada baseada no *proxy* utiliza um *cache* intermediário entre o servidor e o cliente. Este *proxy* pode requisitar arquivos a ser buscados no servidor, ou o servidor pode fazer *push* de alguns arquivos para o *proxy*. Estes dois esquemas

aumentam a banda necessária. A vantagem que se apresenta com relação ao *prefetching* no *proxy* é a proximidade com os clientes. Se comparado ao *prefetching* baseado no cliente, esta abordagem pode ser dita mais eficiente, pois atende a vários clientes, podendo realizar predições baseadas em perfis de acesso de diferentes usuários. Isto é, um documento pode ser buscado antecipadamente e servir para mais de um usuário.

Uma pesquisa realizada por (KROEGER; LONG; MOGUL, 1997) investiga a performance da realização de busca antecipada de informação entre *proxy* e servidor. Como resultado foi apresentado que se forem combinados *caching* perfeito e busca antecipada perfeita a redução sentida pelo cliente é de pelo menos 60% em clientes com banda larga.

7.1.3. Busca Antecipada Baseada no Servidor

Uma busca antecipada baseada no servidor, todo o mecanismo de busca se encontra no servidor. Estas abordagens acabam com os problemas citados nas outras duas. Não há aumento no consumo de banda e os problemas com manutenção da coerência neste caso são mínimos. Neste tipo de abordagem não existem novos protocolos de comunicação entre cliente e servidor, não havendo desperdício de banda ou de recursos do servidor com mensagens extra. Uma grande vantagem é com relação aos mecanismos de checagem de validade dos documentos, que são geralmente nativos do sistema operacional.

As técnicas de busca antecipada de informação no caso de se localizarem no servidor permite que as decisões sejam tomadas não com base nos perfis de acesso de usuários, mas com base na popularidade dos documentos. O único problema é que o *prefetching* é feito de forma geral, e certos usuários (ou grupos de usuários) podem não ser levado em conta.

Zukerman, Albrecht e Nicholson (2001) usa técnicas de IA para prever as requisições do usuário. Seu algoritmo é implementado como uma variação das cadeias de Markov e utiliza os logs de acesso passados como conjunto de treinamento. Esta abordagem se baseia em seguir os passos encontrados em padrões de acesso de usuários. O único problema desta abordagem é a necessidade de muitos cliques do usuário para que o algoritmo aprenda seu padrão.

Para tentar resolver este problema Safronov e Parashar (2001) propõe uma técnica de *Page Rank* baseada no servidor. Seu algoritmo utiliza informações sobre a estrutura de *links* das páginas e dos acessos do usuário corrente e anteriores para realizar a

predição. A abordagem se diz eficiente para acessar clusters e pode reagir imediatamente a mudanças na estrutura de *links* da página. Foram realizados experimentos que apresentaram uma boa performance do algoritmo, chegando a casos de 90% de *hit ratio*. As experiências mostraram também que os acessos a páginas dentro de um mesmo cluster estão entre 15% e 40% de todos os acessos.

Outra proposta baseada no servidor é encontrada em (TAVANAPONG, 2001). Sua abordagem se baseia na hipótese de que, quando um usuário visita um *Web site*, ele geralmente navega por várias páginas antes de visitar outro *site*. O servidor constrói um mapa de co-referências aplicando técnicas de mineração nos seus arquivos de *log*. Tendo isso, quando recebe uma solicitação de algum usuário, o servidor decide quais páginas (descendentes da requisitada) devem ser enviadas ao cliente. As páginas são escolhidas de acordo com os valores de co-referência e um limiar (que pode ser enviado pelo cliente).

7.1.4. Busca Antecipada Baseada no *Proxy* e no Servidor

Uma abordagem que busca resolver os problemas das buscas antecipadas baseadas no *proxy* e no servidor, é introduzida em (CHEN; ZHANG, 2001) através da criação de uma técnica baseada tanto no *proxy* como no servidor. A idéia principal atacada por Chen, é que com a existência de *proxies* entre o cliente e o servidor, o servidor não consegue mapear o comportamento dos usuários. Como resultado foi obtido um aumento na taxa de *hits* utilizando a técnica proposta, quando comparado à abordagem baseada no *proxy*, e resultados semelhantes se comparado à abordagem baseada no servidor.

7.2. Classificação de Acordo com a Estratégia de Predição

A principal questão da busca antecipada de informação está em decidir o quê e quando buscar. Há um grande número de situações de pré-busca implementadas no contexto *Web*, e as diversas propostas de métodos e mecanismos encontrados nestas pesquisas podem ser, em geral, agrupadas da seguinte forma (EL-SADDIK; GRIMODZ; STEINMETZ, 1998): (i) pré-busca não estatística, (ii) pré-busca estatística baseada nos acessos a servidores e (iii) pré-busca baseada nas preferências pessoais do usuário.

7.2.1. Busca Antecipada Baseada em Preferências Pessoais do Usuário

São os mecanismos que implementam formas dos clientes escolherem quais informações querem receber. Existem dois segmentos de implementação. No primeiro, conhecido como tecnologia *Push*, os usuários inscrevem-se em canais de interesse, estabelecendo um perfil de opções. Quando o programa é executado, o mecanismo de busca contata o servidor especificado pelos canais de informações e adquire todos os objetos necessários para que o usuário possa navegar em modo desconectado. O usuário não tem que requisitar ou buscar manualmente a informação, ou seja, as informações não são especificadas pelo usuário e, não se pode avaliar sua real utilização. Um aspecto comum a estes tipos de mecanismos é que há rápida atualização do conteúdo destes canais, provocando um dos seus principais fatores apontados como efeito colateral. Quando este mecanismo se tornou popular, chegou a congestionar redes corporativas com muitos usuários desta tecnologia, porque, além de pré-buscar as informações, verifica constantemente se há atualizações no conteúdo.

No segundo segmento, normalmente chamado de busca determinística ou busca informativa, a busca também é configurada de forma estática pelos usuários, contudo, é mais específica que o anterior. Fazendo uma analogia, no primeiro segmento o usuário especifica quais os canais assiste numa TV e, no segundo, quais os programas de TV. Na busca determinística, o usuário configura quais os documentos quer receber dentre os que poderiam ser acessados manualmente através de um navegador (por exemplo, pode determinar o recebimento diário das páginas das seções de esporte e política de um jornal on-line, deixando-as disponíveis no *cache* para que se possa acessá-las desconectado). Ela representa o tipo mais conservativo de busca antecipada, já que, frequentemente, ocasionam pouca ou nenhuma sobrecarga na largura de banda, embora tenha um escopo de uso limitado, já que o usuário precisa saber e determinar o que será pré-buscado (OLIVEIRA, 2002a).

Um sistema que utiliza este tipo de mecanismo de predição é a *cache* Venus do sistema de arquivos Coda (SATYANARAYANAN, 2002). A Venus opera em um dos três estados: *hoarding*, *emulating* ou *reintegrating*. A *cache* se encontra no estado *hoarding* quando a conexão com os servidores é boa. Neste estado ocorre a preparação para o caso de uma possível desconexão. Esta preparação se dá através do armazenamento de objetos que possivelmente serão úteis ao usuário. A decisão dos arquivos a serem armazenados é realizada seguindo as informações de um banco de dados contendo o caminho dos objetos de interesse do usuário. Estas informações são

fornecidas pelo próprio usuário e ordenadas através de um algoritmo LRU, priorizando os objetos acessados mais recentemente. Quando a conexão é perdida, a *cache* passa para o estado *emulating*. Neste estado todas as modificações nos objetos são guardadas em um arquivo de *log*. Quando a conexão volta ou quando a conexão é fraca, ela passa para o estado *reintegrating*, no qual são feitas atualizações necessárias, junto ao servidor.

7.2.2. Busca Antecipada com Predição Não-Estatística

São os mecanismos de predição menos sofisticados. Normalmente, buscam todas as páginas referenciadas num documento HTML requisitado pelo cliente, executando uma busca dita agressiva. Nos primórdios da *Web*, quando a quase totalidade dos documentos era simples, sem muitos objetos agregados, este tipo de busca apresentava maior eficácia, mas ainda era insuficiente, por não executar uma predição especulativa. Ela resultava em melhora na latência percebida pelo usuário, mas se tornaria inviável se fosse genericamente implementada, dada a sobrecarga na infra-estrutura total da *Web*.

Atualmente, alguns produtos comerciais implementam esta opção, todavia com alguma maneira de se controlar determinadas opções, tais como escolher quais os tipo de objetos que se quer buscar antecipadamente. Outros já começam a implementar algum fator especulativo, contudo, ainda simples e ineficientes. Vale notar que estas implementações são normalmente úteis em conexões clientes feitas por *modems*, em que o aproveitamento o tempo ocioso é de grande utilidade.

7.2.3. Busca Antecipada com Predição Estatística

Este tipo de estratégia de busca antecipada se baseia nos acessos passados dos usuários para prever seus acessos futuros. Isto é feito observando-se a probabilidade de que uma mesma sequência de passos executada anteriormente volte a ser seguida. As técnicas desta estratégia se baseiam no fato de que usuários não acessam os documentos aleatoriamente e, embora o padrão de acesso dos usuários não possa ser determinado exatamente, é possível ter uma idéia de quais serão os próximos documentos que serão acessados após o documento corrente. Normalmente, esta estratégia de busca é chamada de especulativa, pois investiga o que deve ser buscado antecipadamente, ao contrário das outras estratégias. Estas técnicas abrangem um grande número de pesquisas, porque visa uma forma eficiente de busca, na qual não há intervenção direta do usuário.

O agente que realmente realiza a busca antecipada é um cliente (*browser* ou *proxy*). Contudo, de acordo com qual parte está mais envolvida para fornecer as informações necessárias para a estatística, os diferentes gêneros de pré-busca especulativa podem ser classificados nas seguintes categorias (KROEGER; LONG; MOGUL, 1997; OLIVEIRA, 2002a):

- *Baseada no Cliente/Proxy*. Baseia-se no comportamento de navegação de usuários específicos em diversos servidores *Web*, sendo a informação relevante reunida no lado cliente. Pode ser feita tanto tendo por base um usuário, como uma comunidade de usuários. Neste modelo, a acurácia da predição é limitada, pois ainda não se têm os dados do servidor global.
- *Baseada no Servidor*. Baseia-se no comportamento de todos os usuários acessando um servidor *Web* específico. O processo central da busca antecipada é realizado no servidor, pois este fornece as informações de acesso dos usuários. A grande vantagem de basear-se no servidor é que o mecanismo de busca aproveita todas as seqüências de referências de um grande número de clientes. Todavia, a desvantagem está na necessidade de freqüentes comunicações entre cliente/*proxy* e os servidores, aumentando a latência.

Embora a união destes dois fatores junte as vantagens de ambas abordagens, a complexidade da implementação a impossibilitaria, uma vez que requer modificações tanto no servidor como no cliente. Resta salientar que, dependendo do limiar de probabilidade, pode-se optar por uma configuração agressiva ou conservativa (Fan; Cao; Jacobson, 1999).

7.3. Medindo a Performance de Sistemas de Busca Antecipada

As maneiras de medir-se a performance de sistemas de busca antecipada normalmente são os mesmos usados para sistemas de *Web caching*. Costuma-se mostrar a eficiência através de *miss ratio*, *hit ratio* e latência média, sendo o *miss ratio* a mais aceita. Porém, muito se questiona a efetividade dessas métricas no contexto da busca antecipada.

Segundo Satyanarayanan (1996) estas métricas não são de todo válidas, pois o custo de um *miss* pode ser muito maior nos casos de estar-se operando com o *cache* em modo desconectado. O momento em que o *miss* ocorre também deve ser levado em conta. Para um usuário um *miss* no início do período de desconexão pode ser menos custoso que um *miss* depois de um certo tempo de desconexão.

Uma medida alternativa, primeiramente sugerida em (SATYANARAYANAN et al., 1993), é a chamada *time to first hoard miss*, medida como o tempo decorrido ou como o número de referências realizadas até o primeiro *miss*. A métrica é interessante, pois quantifica trabalho que o usuário pode realizar antes de uma falha.

Kuenning (KUENNING et al., 2002; KUENNING; POPEK, 1997) diz que encontrar uma maneira de medir e comparar a performance de sistemas de busca antecipada de informações é uma das maiores dificuldades dentro da área. Ele também critica o uso de *miss ratio*, dizendo que esta não é a métrica apropriada, uma vez que nem todos os *misses* são iguais. Com relação ao *time to first hoard miss*, Kuenning diz que a métrica é muito dependente do tamanho do *working set* do usuário e que se torna difícil utilizá-la na comparação de duas estratégias, pois seria necessário que o usuário realizasse a mesma tarefa duas vezes.

Para tentar resolver estes problemas, Kuenning e Popek (1997) sugeriram a métrica intitulada *miss-free hoard size (MFHS)* que é definida como o tamanho necessário para se ter certeza de que não haverão *misses* durante um determinado espaço de tempo. Por exemplo, se LRU estivesse sendo utilizado para ordenar os objetos a serem buscados, o MFHS seria calculado da seguinte maneira: (i) Todos os arquivos seriam ordenados segundo o LRU; (ii) todos os arquivos requisitados durante o período de tempo determinado são marcados; (iii) o último arquivo marcado é localizado na lista; e (iv) o tamanho de todos os arquivos entre o início da lista e o arquivo marcado é somado. Segundo os autores esta métrica apresenta a vantagem de não necessitar de parâmetros, ser calculada utilizando *traces* de referência e refletir o comportamento que o usuário deseja (trabalhar em modo desconectado, sem tomar consciência de que está utilizando um subconjunto restrito de informações).

7.4. Considerações Finais

Este capítulo apresentou a busca antecipada de informações como forma de incrementar a performance dos sistemas de *Web caching*. Foram apresentadas duas formas de classificação dos mecanismos de busca antecipada. Na primeira forma, os

mecanismos são organizados segundo a localização do *Web caching* dentro do contexto *Web*. Foram mostradas alternativas de realização de busca antecipada no cliente, no *proxy* e no servidor, e expostas suas vantagens e desvantagens. Através da análise de cada uma das classes fica claro que a decisão da localização depende, mais uma vez, de uma boa definição dos objetivos da aplicação a ser desenvolvida.

A outra classificação é baseada na maneira com que é feita a decisão dos objetos a serem buscados antecipadamente. Esta classificação é de grande importância, pois um dos maiores problemas nos sistemas que usam busca antecipada é a decisão de quais objetos e quando devem ser buscados. Esta definição é totalmente dependente do domínio da aplicação a ser desenvolvida.

Embora seja eficiente, a busca antecipada de informações na *Web* tem alguns efeitos colaterais, o principal deles é o acréscimo do tráfego na rede. Algumas propostas como a de Fan, Cão e Jacobson (1999) buscam resolver este problema. O fato é que os resultados obtidos oferecem aos usuários uma latência aparente menor.

Por último, foi apresentada uma discussão sobre as maneiras de medir a performance dos sistemas de busca antecipada de informação. Duas novas abordagens (*time to first hoard miss* e MFHS) foram apresentadas, visto que as métricas existentes não mapeiam o custo existente na ocorrência de possíveis *misses*.

8. CONCLUSÃO

Este trabalho trouxe uma revisão bibliográfica de técnicas de *Web caching* e de *prefetching*, afim de que seja realizada a definição de uma estratégia de busca antecipada de informação aliada a um sistema de *Web caching* para aumentar a disponibilidade de sistemas que operam em modo desconectado ou semiconectado. Foram encontradas várias pesquisas relacionadas a estes tópicos de forma isolada (CHANKHUNTHOD et al., 1996; KROEGER; LONG; MOGUL, 1997; ARLITT; FRIEDRICH; JIN, 1999; RODRIGUEZ; SPANNER; BIRSACK, 2001) e algumas que buscam uni-las (PADMANABHAN; MOGUL, 1996; FAN; CAO; JACOBSON, 1999; DAVIDSON, 2001) justamente a fim de aumentar a disponibilidade de sistemas distribuídos.

Os sistemas de *Web caching* foram criados com intuito de reduzir a latência sentida pelo usuário final. Acabaram por trazer várias vantagens a todos os agentes envolvidos na *Web*. Uma das vantagens trazidas pelo uso de *cache* na *Web* é o aumento da disponibilidade dos sistemas, que escondem certos problemas que podem ocorrer entre *cache* e servidor. O ponto onde se encontra o *cache* também foi analisado. Esta análise foi feita de modo a mostrar vantagens e desvantagens contidas em cada uma das abordagens. Outro ponto que foi abordado foi a forma com que se pode criar cooperação entre *proxy caches* a fim de aumentar a performance de um sistema. Com relação a sistemas móveis, que podem operar em modo desconectado, a melhor alternativa seria uma hierarquia, constituída de, ao menos, um *proxy* institucional (ou regional) e um *proxy* no cliente. O *proxy* regional ficaria responsável por guardar uma grande parte dos dados que seriam repassados segundo uma ordem de prioridades para o *proxy* do cliente quando este estivesse conectado.

Uma revisão mais profunda foi realizada com relação aos métodos de substituição de objetos contidos na *cache*. O funcionamento de várias estratégias encontradas na literatura foi apresentado. A decisão da melhor política a ser utilizada não é uma tarefa

trivial, sendo necessário traçar detalhadamente os objetivos do sistema que está se desenvolvendo. Ficou claro que, em certos casos, se faz necessária a criação de uma estratégia específica para o sistema unindo várias estratégias diferentes ou de uma função que ordene os objetos de forma ideal. Dentro deste tópico, se for analisado o caso dos dispositivos que operam em modo desconectado, pode-se dizer que alternativas que levem em conta o tamanho dos objetos são as preferidas, uma vez que o espaço de armazenamento neste tipo de dispositivos são um fator limitante. Talvez uma técnica próxima do *HYPER-G*, mas baseada em função, seja adequada, pois leva em conta o tamanho, e ordena os objetos usando LRU e LFU.

Por último foi apresentada uma revisão bibliográfica das técnicas de busca antecipada de informações encontradas. Uma conclusão tirada desta revisão foi que apesar dos sistemas diminuírem a latência sentida pelo usuário eles têm como consequência perdas com relação à latência da rede. Porém a utilização da busca antecipada é de suma importância para sistemas que operam em modo desconectado, onde a busca antecipada tem como meta indicar os dados que o usuário necessitará durante o próximo período de desconexão. Neste trabalho são apresentadas algumas estratégias de predição existentes, afim de evidenciar suas vantagens e desvantagens. Mais uma vez não foi possível escolher o melhor mecanismo, visto a dependência dos mecanismos com os objetivos do sistema.

Através da análise realizada verificou-se a possibilidade real de resolver o problema da operação em modo desconectado através da combinação de técnicas de *Web caching* e *prefetching*. Com base no conhecimento adquirido com a execução deste trabalho, se torna possível definir os objetivos gerais e as etapas necessárias ao desenvolvimento de um sistema de *Web caching* e *prefetching* com prioridades. Os passos a serem seguidos são, basicamente, a definição dos objetivos do sistema a ser desenvolvido, seguida da realização de uma análise quantitativa das técnicas aqui apresentadas e implementação baseada nos resultados da análise.

9. BIBLIOGRAFIA

- ABRAMS, M., et al. *Caching Proxies: Limitations and Potentials*. In: 4th International World Wide Web Conference, **Proceedings...** Boston, 1995. p.119-133.
- ARLITT, M. F. et al. Evaluating Content Management Techniques for *Web Proxy Caches*. **ACM SIGMETRICS Perform Evaluation Review**, v.27, n.3, p. 3-11, Mar 2000.
- ARLITT, M.; FRIEDRICH, R.; JIN, T. **Workload characterization of a Web proxy in a cable modem environment**. Technical Report HPL-1999-48, Hewlett Packard Labs, 1999.
- BOLOT, J.; HOSCHKA, P. Performance Engineering of the World Wide Web: Application to Dimensioning and *Cache* Design. In: 5th WWW Conference, **Proceedings...** 1996. p. 1397-1405.
- CAO, P.; IRANI, S. **Cost Aware WWW Proxy Caching algorithms**. In: USENIX Symposium on Internet Technologies and Systems, 1997
- CARP. *Cache Array Routing Protocol*, Disponível em: <http://www.microsoft.com/technet/Proxy/technote/prxcarp.asp>. Acesso: jan 2004
- CATE, V. **Alex – A Global File System**. In: USENIX File System Workshop, 1992.
- CHANG, C.Y.; MCGREGOR, T.; HOLMES, G. **The LRU* WWW Proxy Cache Document Replacement Algorithm**. In: Asia Pacific Web Conference, 1999.
- CHANKHUNTHOD, et al. **A Hierarchical Internet Object Cache**. In: USENIX Technical Conference, San Diego, 1996.
- CHEN, X.; ZHANG, X. Coordinated data *prefetching* by utilizing reference information at both *proxy* and *Web* servers. **ACM SIGMETRICS Performance Evaluation Review**, v. 29, n. 2, p. 32-38, set 2001
- CROVELLA, M.; BARFORD, P. **The Network Effects of Prefetching**. Proceedings of IEEE Infocom, 1998.
- CUNHA, C. R.; JACCOUD, C. F. B. **Determining WWW user's next access and its application to pre-fetching**. In: Proceedings of The second IEEE Symposium on Computers and Communications, 1997.
- DAVISON, B. D. **Assertion: Prefetching with GET is Not Good**. Proceedings of the 6th Interncional Workshop *Web Caching* and Content Distribution. Boston, jun. 2001.
- DIX, A.; BEALE, R. **Information requirements of distributed workers**. In: Remote Cooperation: CSCW Issues for Mobile and Teleworkers, Nova York: Springer, p. 113-144, 1996.

- EBLING, M. R.; JOHN, B. E.; SATYANARAYANAN, M. The Importance of Translucence In Mobile Computing Systems, **ACM Transactions on Computer-Human Interaction (TOCHI)**, v. 9, n. 1, p. 42-67, mar 2002.
- EL-SADDIK, A.; GRIWODZ, C.; STEINMETZ, R. **Exploiting User Behaviour in Prefetching WWW Documents**. Internacional Workshop on Interactive Distributed Multimedia Systems and Telecommunication Services, sept. 1998.
- FAN, L.; CAO, P.; JACOBSON, Q. **Web Prefetching Between Low-Bandwidth Clients e Proxies: Potential and Performance**. Proceedings of the Sigmetrics, 1999.
- GWERTZMAN, J.; SELTZER, M. I. **World Wide Web Cache Consistency**. In: USENIX Annual Technical Conference, 1996. p. 141-152
- JIANG, Z.; KLEINROCK, L. An Adaptive Network *Prefetch* Scheme. **IEEE Journal on Selected Areas in Communications** 1998, 1998.
- KRISHNAMURTHY, B.; WILLS, C. E. **Study of Piggyback Cache Validation for Proxy Caches in WWW**. In: USENIX Symposium on Internet Technology and Systems, 1997.
- KRISHNAMURTHY, B.; WILLS, C. E. **Piggyback Server Invalidation for Proxy Cache Coherency**. In: 7th WWW Conference, 1998.
- KRISHNAMURTHY, B.; REXFORD, J. **Redes para a Web**. Rio de Janeiro: Campus, 2001. 651 p.
- KROEGER, T. M.; LONG, D. D. E.; MOGUL, J. C. **Exploring the bounds of Web latency reduction from caching and prefetching**. In: USENIX Symposium on Internet Technologies and Systems, 1997. p. 13–22.
- KUENNING, G. H.; POPEK, G. J. **Automated Hoarding for Mobile Computers**. ACM Symposium on Operating Systems Principles, 1997.
- KUENNING, G. H.; MA, W.; REIHER, P.; POPEK, G. J. **Simplifying Automated Hoarding Methods**. 5th ACM international workshop on Modeling analysis and simulation of wireless and mobile systems, 2002.
- MATEUS, G. R.; LOUREIRO, A. A. F. **Introdução à Computação Móvel**. Rio de Janeiro, DCC/IM, COPPE/UFRJ, 1998. 189p.
- MENAUD, J. M.; ISSARNY, V. BANATRE, M. Improving Effectiveness of *Web Caching*. **Recent Advances in Distributed Systems**. Berlim: Springer-Verlag, v.1752, p. 375-401, 2000.
- OLIVEIRA, D.X.T., **Análise e Avaliação de Técnicas de Pré-Busca em Caches para Web**. Monografia (Mestrado em Ciências da Computação e Matemática Computacional) – ICMC, USP, São Carlos, 2002a.
- OLIVEIRA, J. A. **Uso de Caches na Web – Influência das Políticas de Substituição de Objetos**. 47 f. Monografia (Mestrado em Ciências da Computação e Matemática Computacional) – ICMC, USP, São Carlos, 2002b.
- PADMANABHAN, V. N.; MOGUL, J. C. **Using predictive prefetching to improve World Wide Web latency**. In: Proceedings of Sigcomm, 1996.
- PITKOW, J.; RECKER, M. **A simple yet robust caching algorithm based on dynamic access patterns**. In: 2nd International World Wide Web Conference. Chicago, 1994

- PODLIPNIG, S.; BÖSZÖRMENYI, L. A Survey of *Web Cache* Replacement Strategies. **ACM Computing Surveys (CSUR)**, v.35, n.4, p.374-398, dez 2003.
- POVEY, D.; HARRISON, J. **A Distributed Internet Cache**. In: 20th Australian Computer Science Conference, Sydney, Australia, 1997.
- RABINOVICH, M.; SPATSCHECK, O. **Web Caching and Replication**. Addison-Wesley, 2002. 400 p.
- RODRIGUEZ, P.; SPANNER, C.; BIRSACK, E. W. Analysis of *Web Caching* Architectures: Hierarchical and Distributed *Caching*. **IEEE/ACM Transactions on Networking**, v.9, n.4, p. 404-418, ago 2001
- SAFRONOV, V.; PARASHAR, M. Optimizing *Web* servers using page rank *prefetching* for clustered accesses. **Information Sciences - Informatics and Computer Science: An International Journal**, v. 150, n. 3-4, p. 165-176, abr 2003.
- SATYANARAYANAN, M.; KISTLER, J. J.; MUMMERT, L. B.; EBLING, M. R.; KUMAR, P.; LU, Q. **Experience with disconnected operation in a mobile computing environment**. In: USENIX Symposium on Mobile and Location-Independent Computing, Cambridge, 1993.
- SATYANARAYANAN, M. **Fundamental Challenges in Mobile Computing**. Annual ACM Symposium on Principles of Distributed Computing. Pensilvânia, 1996
- SATYANARAYANAN, M. The Evolution of CODA, **ACM Transactions on Computer Systems (TOCS)**, v. 20, n. 2, p. 85-124, mai 2002.
- VAKALI, A. **LRU-Based Algorithms for Web Replacement**. In: 1st International Conference on Electronic Commerce and *Web* Technologies, 2000.
- VIDAL, N.R. **Everware & Summware: Um Sistema de Gerência de Hiperdocumentos Adaptativos para Dispositivos Móveis**. Trabalho de Diplomação (Engenharia da Computação) – UFRGS, Porto Alegre, 2003.
- WANG, J. A survey of *Web Caching* for Internet. **ACM SIGCOMM Computer Communication Review**, v.26, n.5, p. 36-46, Out 1999.
- WESSELS, D. **Web Caching**. 1.ed., O'Reilly and Associates, 2001. 318 p.
- WESSELS, D.; CLAFFY, K. ICP and the Squid *Web Cache*. **IEEE Journal on Selected Areas in Communication**, v.16, n.3, abr 1998.
- WILLIAMS, S. et al. **Removal Policies in Network Caches for World Wide Web documents**. In: ACM SIGCOMM, 1996.
- WOOSTER, R.; ABRAMS, M. **Proxy Caching that Estimates Page Load Delays**. In: 6th WWW Conference, 1997.
- ZUKERMAN, I.; ALBRECHT, W.; NICHOLSON, A. **Predicting user's request on the WWW**. In: Proceedings of the Seventh International Conference on User Modeling, 1999.